

Advice: The Witness

Client: Nightfall Interactive

Made by: David Eslava, Fontys ICT student

Version 0.2, 24 September 2026

Table of contents

1. What is this document about	2
2. What is important for this project	2
3. Decision 1: engine and programming language	2
3.1 The options	2
3.2 Good and bad points	3
3.3 My advice	3
4. Decision 2: how the enemies move and decide where to go	4
4.1 How the guards move around the level	4
4.2 How the guards decide where to go	5
4.3 How The Hunter decides where to go	5
4.4 My advice	6
5. Summary	6
6. Sources	6

1. What is this document about

Before I start building, I need to make two decisions. First, I have to choose which engine and programming language to use for the game. Second, I need to decide how the enemies will move and choose their paths.

In this document, I compare the options for both decisions and share my advice with **Nightfall Interactive**. The requirements are listed in the Analysis. Details about how I will build the game are in the Design document.

2. What is important for this project

I am working alone and have four weeks to finish the project. Since this is my first time using a game engine, I need to learn the tool as I build the game. This is the main challenge for me.

The game will be 2D and viewed from above. I will need to handle wall collisions, create animations in eight directions, set up a camera, and design some simple screens. The tools also have to be free, because there is no money for licences.

I already know some Python, so if the engine's language is similar, I will learn faster. Also, the client wants an enemy that can learn. Whatever I choose now should not make it impossible to build The Hunter later.

3. Decision 1: engine and programming language

3.1 The options

	Godot 4 + GDScript	Unity + C#	GameMaker + GML	Pygame + Python
Price	Free, open source	Free until you earn a certain amount, then you pay	Free for non-commercial use, you pay to sell the game	Free, open source
Easy to learn for a first engine?	Yes, the editor is small	No, I would learn C# and a big editor at the same time	Yes	Easy to start, but hard when the game gets bigger
2D tools included (scenes, collisions, animations, camera)	Yes	Yes	Yes	No, I have to program them myself
Language similar to Python	Yes, GDScript is based on Python	No	No	It is Python
Machine learning libraries	No	Some (ML-Agents)	No	Yes, a lot

3.2 Good and bad points

At first, I thought about using Pygame because it uses Python, and most reinforcement learning tutorials I watched also use Python. But after watching two tutorials on making games with Pygame, I changed my mind. Pygame is a library, not a full engine. I would have to program collisions, animations, scenes, and the camera myself, which would leave almost no time for the actual game in four weeks.

Unity is the most widely used engine in the industry and even has its own machine learning tools. It is a good choice for experienced studios, but for me, it would mean learning C# and a large editor at the same time. That is too much for a first project.

GameMaker is good for 2D and easy to learn. The problem is that its language only works inside GameMaker, and you have to pay to publish a game. Godot gives me the same 2D tools for free.

Godot has everything I need, and GDScript is very similar to Python. Before deciding, I followed a beginner tutorial in Godot (the `first-game` project in my Realisation folder) and made a simple project with a moving character, a patrolling enemy, and a zone that defeats the player. This gave me confidence. The downside is that GDScript does not have machine learning libraries. However, for The Hunter, I only need a table with numbers (see Decision 2), and GDScript can handle that without extra libraries.

3.3 My advice

Make the game in **Godot with GDScript**. It is free and it has all the 2D tools the game needs. I can also use what I already know from Python.

The only thing we lose are Python's machine learning libraries, and that would only be a problem if The Hunter ever needs a neural network.

4. Decision 2: how the enemies move and decide where to go

There are two types of enemies in the game. The **guards** patrol and react when they see the player. **The Hunter** is the main focus for the client because he gets better at catching the player over time.

When an enemy moves, there are actually two different questions. The first is how it gets to a place. It has to walk around the bar and the tables, not into them (requirement M-07), and this is called **pathfinding**. The second question is where it wants to go: the next patrol point, the player, or the last place it saw the player. For the guards a few simple rules answer this. For The Hunter, this is the part that has to learn.

4.1 How the guards move around the level

	Straight line to the target	Pathfinding (NavigationRegion2D + NavigationAgent2D)	My own pathfinding on a grid (A*)
What is it	The guard walks straight to the target and slides along things in the way	Godot finds the shortest way around walls and furniture	I program the search for the route myself
Walks around obstacles?	No, it gets stuck in corners	Yes	Yes
Already in Godot	Yes	Yes	No
Time to build	Minutes	A few hours, mostly to learn how to set it up	Days

Most beginner tutorials use a straight line. That works until there is a table between the guard and the player. Then the guard keeps walking into the table, it looks broken, and the game is not tense anymore. I could program my own A*, and I would learn a lot from it, but Godot already has pathfinding and it is tested. I prefer to spend my time on the stealth part.

My advice: the guards use **the pathfinding that Godot already has**. I mark the floor where they can walk with a NavigationRegion2D. Every guard gets a NavigationAgent2D that finds the way around the furniture.

4.2 How the guards decide where to go

Pathfinding only tells the guard *how* to get somewhere. The guard still needs to decide *where* to go. Here are the options for that:

	Simple rules (states)	Q-learning with a table	Deep reinforcement learning
What is it	A few rules: patrol, chase the player when you see them, stop when you catch them	The enemy has a table that says how good every move is in every situation, and it updates the table after every move	A neural network learns the same thing from a lot of examples
Learns from the player?	No	Yes	Yes
Works inside Godot without other programs	Yes	Yes, it is only a table	No, it needs Python and extra libraries
Can I explain why it did something?	Yes	Yes, I can read the table	Very difficult
Time to build	Hours	Days, with the training	Weeks

Simple rules are enough for the guards, which is also what the game needs. The player should be able to learn the guards' routes to sneak past them, so the guards need to be predictable. They also serve as a comparison, since The Hunter is only interesting next to enemies that do not learn. With pathfinding, the guards will still move naturally through the club.

4.3 How The Hunter decides where to go

For The Hunter, pathfinding alone is not enough. Pathfinding always finds the shortest route, so The Hunter would be perfect from the start and would never improve. This would leave nothing to measure, and measuring improvement is exactly what the client wants.

Deep reinforcement learning is what they use in the "A.I. Learns to Drive" and Snake videos I watched. It needs much more training and it works outside Godot. When it does something strange, it is also very hard to know why. The Hunter only chooses between four moves on a small map, so this is too much for the problem.

Q-learning with a table is the simplest method that actually learns. The Hunter looks at the situation, picks one of four moves, gets or loses points, and records the result in a table. After many tries, the best moves have higher scores. The Hunter's entire "brain" is a table of about 1,500 numbers, which I can open and read, making it easy to check the results.

4.4 My advice

For the guards, I recommend using Godot's pathfinding for movement and a few simple rules to decide where they go. For The Hunter, I suggest using Q-learning with a table.

I will build the guards for this delivery because the stealth game needs to work first. I will fully design The Hunter now, and he will be built in the next iteration, as explained in the Analysis.

5. Summary

Decision	My advice	Main reason	What we lose
Engine and language	Godot 4 with GDScript	It is free, has the 2D tools and looks like Python	The machine learning libraries of Python
How the guards move	Godot's pathfinding (NavigationAgent2D)	They walk around the furniture, not into it (M-07)	Some time to learn the setup. I also do not program the route search myself
What the guards decide	Simple rules: patrol, chase, catch	Easy to read, so the player can learn their routes	They never surprise the player
The Hunter	Q-learning with a table, designed now, built in the next iteration	The simplest method that really learns, and I can explain it	He can only learn simple situations, not complex tactics

6. Sources

All my sources are in section 7 of the Analysis. These are the ones that helped me most with these two decisions.

To see what it means to make a game with Pygame, I watched [How to Make a Game in Python](#) and [Making a Game in Python with No Experience](#) on YouTube. I followed [How to Make a Video Game - Godot Beginner Tutorial](#) (YouTube) to make the first-game test project.

For Q-learning with a table I used [Q Learning Intro/Table - Reinforcement Learning.p.1](#) by sentdex on YouTube, and [A Beginner's Guide to Q-Learning](#) on Medium. [Deep Reinforcement Learning Applied to the Snake Video Game](#) and [A.I. Learns to Drive](#) (both YouTube) showed me what deep reinforcement learning needs.

I checked all of them between 1 and 22 September 2026.